Star Defender Engine Architecture

EF101-P05 Documentation — Core Systems & Logic Matrices

> **The Conceptual Framing (The Digital Flipbook):**
> Star Defender contains zero external asset files (no `.png` or `.mp3` files). Instead, it relies on real-time procedural generation. The Python engine synthesizes raw sound waves in the system memory, mathematically draws vector shapes on a logical grid, and completely erases and redraws the screen 60 times every second. This creates the continuous illusion of interactive movement.

MATRIX 1: ENGINE FUNDAMENTALS

These four mechanisms form the non-negotiable core of the game. If any of these systems are removed or fail, the application cannot run.

Core Concept	Architectural Purpose	Implementation / Syntax Target

The Game Loop	Maintains the heartbeat of the application. An infinite loop that forces the engine to listen for inputs, calculate physics, and render graphics exactly 60 times per second.	`Game.run()`
		`1. handle_events()`
		`2. update(dt)`
		`3. draw()`

Delta Time (dt)	Hardware normalization. Prevents the game from running faster on powerful PCs and slower on older laptops by scaling movement to the exact fraction of a second passed.	`position += speed * dt`

Coordinate System	Maps logical entities to physical screen pixels. PyGame reverses standard mathematical quadrants: the origin is at the top-left of the window.	`Origin: (0,0)`
		`+X moves Right`
		`+Y moves Down`

Collision Engine	Determines physical interaction by calculating whether the mathematical bounding boxes of two independent objects overlap in 2D space.	`Rect.colliderect(other_rect)`

MATRIX 2: OBJECT-ORIENTED BLUEPRINTS

The code is structured using isolated classes to ensure clean memory management. Each class handles a single, distinct responsibility.

Blueprint (Class)	Role & Responsibility	Primary Dependency
Game	**The Director.** Manages the central state machine (Title → Playing → Game Over). Owns the display surface and orchestrates the updates of all other entities.	`pygame.display` `pygame.event`
Player & Enemy	**The Actors.** Independent nodes that track their own speed, position, and hit points. They self-manage their boundaries and cooldown timers.	`pygame.Rect` `pygame.draw`
SoundManager	**The Voice.** Procedurally generates audio by writing mathematical sine and square waves into memory buffers at runtime.	`pygame.sndarray` `math.sin`
SaveData	**The Memory.** Reads and writes local data to persist multi-user profiles and high scores across different sessions.	`json` `os.path`

MATRIX 3: ADVANCED DATA INTEGRITY

Specialized logic implemented to ensure a professional user experience and prevent data corruption.

Mechanic	Problem Solved	Implementation / Syntax Target
Atomic Saving	If a PC crashes exactly while saving a high score, a standard file overwrite will corrupt the JSON. Atomic saving writes to a temporary file first, only overwriting the master file if the write succeeds.	`write to savedata.json.tmp` `os.replace(.tmp, .json)`
Entity Filtering	If bullets fly off the screen forever, the computer will eventually run out of RAM. The engine scrubs lists every frame to delete dead or off-screen objects.	`bullets = [b for b in bullets if b.active]`